

Pearls AQI Predictor: Automated CI/CD Pipeline with GitHub Actions

Milestone 6 Technical Report

Ayesha Salahuddin
Pearls AQI Predictor Project
Karachi, Pakistan

Abstract—This report describes the automation of the Pearls AQI Predictor data and model pipelines using GitHub Actions as a serverless continuous integration and continuous deployment (CI/CD) platform. Two scheduled workflows were implemented: a feature pipeline that runs every hour to collect live air quality data into the Hopworks Feature Store, and a training pipeline that runs daily to retrain the forecasting model and update the Model Registry. API credentials are managed securely using GitHub encrypted secrets. Both workflows were tested successfully, establishing a fully automated, hands-free machine learning system consistent with the project’s serverless design goal.

Index Terms—CI/CD, GitHub Actions, automation, serverless, MLOps, scheduled workflows

I. INTRODUCTION

A central requirement of the Pearls AQI Predictor is that the entire system operates automatically, without manual intervention, on a 100% serverless stack. While earlier milestones produced working scripts for feature engineering and model training, these still had to be executed by hand. The purpose of this milestone is to automate their execution on a schedule using a CI/CD tool, so that fresh data is continuously collected and the model is regularly retrained.

GitHub Actions was selected as the automation platform. It is free for public repositories, integrates directly with the project’s existing GitHub repository, and runs jobs on cloud-hosted machines, requiring no server administration. This satisfies the serverless objective of the project.

II. WORKFLOW DESIGN

Two independent workflows were defined as YAML files within the repository’s `.github/workflows` directory.

The **feature pipeline workflow** executes the feature collection script on an hourly schedule. On each run it provisions a clean Ubuntu environment, installs Python and the project dependencies, and runs the script that fetches the latest air quality reading and inserts the engineered features into the Hopworks Feature Store.

The **training pipeline workflow** executes the training script on a daily schedule. It reads the accumulated historical features from the Feature Store, retrains the candidate models, selects the best by R-squared, and saves it to the Model Registry.

Both workflows use a cron schedule for automatic execution and additionally expose a manual trigger

(`workflow_dispatch`), which allows on-demand runs for testing and demonstration. Table I summarizes the schedules.

TABLE I
AUTOMATED WORKFLOW SCHEDULES

Workflow	Schedule
Feature pipeline	Every hour (cron 0 * * * *)
Training pipeline	Daily at 02:00 UTC (cron 0 2 * * *)

III. SECURE CREDENTIAL MANAGEMENT

The pipelines require credentials for the AQICN API and the Hopworks platform. Hardcoding these in the source code would expose them publicly, which is a serious security risk. To avoid this, the credentials were stored as encrypted repository secrets in GitHub and injected into the workflows as environment variables at runtime. The scripts read these values using environment-variable lookups rather than literal keys, ensuring that no secret appears in the codebase or in logs.

IV. DEPENDENCY RESOLUTION

Because each workflow run begins in a clean environment, all dependencies must be declared explicitly in a `requirements.txt` file. During testing, the Hopworks client was found to require additional components for writing data to the Feature Store, namely the Apache Arrow and Kafka client libraries. These were resolved by installing the Hopworks client together with its Python extras, which bundle the required components. This iterative debugging of the cloud environment is a normal part of building a reproducible automated pipeline and highlights the difference between a local development setup and a clean automation environment.

V. TESTING AND VERIFICATION

Both workflows were executed manually using the GitHub Actions interface to verify correctness before relying on the schedule. The feature pipeline completed successfully, fetching the current AQI reading and inserting it into the Feature Store. The training pipeline completed successfully, retraining the model and updating the Model Registry. With manual verification confirming correct behaviour, the scheduled triggers will execute the same workflows automatically at their defined intervals.

VI. CONCLUSION

This milestone established a fully automated CI/CD pipeline for the Pearls AQI Predictor using GitHub Actions. The feature pipeline now collects data hourly and the training pipeline retrains the model daily, both without manual intervention and with credentials managed securely. This completes the automation layer of the system and provides a continuously updated model and feature store for the forecasting dashboard developed in the next milestone.